

Mini-HPC and Hybrid HPC-Big Data Clusters

Hashem Yasser Youssif 231000770 Malak Ahmed Mohamed 241002087

Youssef Rabeay Younes 231001982

1. Virtual Machines Creation

VirtualBox was used to create three Ubuntu Server virtual machines named: master, worker1, and worker2.

Each virtual machine was configured with:

- 4096 MB RAM
- 20 GB Storage
- 2 CPUs
- Ubuntu Server 20.04 LTS

During installation:

- OpenSSH Server was enabled.
- User account 'student' was created.
- Server names were configured individually.

2. Configure Networking

A Host-Only Adapter network was configured in VirtualBox. Static IP addresses were assigned to all nodes.

Master Node:

192.168.56.10

Worker1:

192.168.56.11

Worker2:

192.168.56.12

The /etc/hosts file was updated on all machines to allow hostname communication. Connectivity was verified using ping commands.

3. SSH Configuration

SSH was configured to allow secure communication between all cluster nodes.

Passwordless authentication was enabled using RSA SSH keys.

The master node copied its public key to all worker nodes using ssh-copy-id.

Ip a :

```
student@master:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:47:2a:67 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fd17:625c:f037:2:a00:27ff:fe47:2a67/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86186sec preferred_lft 14186sec
    inet6 fe80::a00:27ff:fe47:2a67/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:29:96:6c brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe29:966c/64 scope link
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:96:71:5a:23 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
student@master:~$ _
```

passwordless SSH ;

```
student@master:~$ ssh student@worker1
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-216-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue 02 Jun 2026 09:52:12 AM UTC

System load:          0.06
Usage of /:           45.8% of 9.75GB
Memory usage:         7%
Swap usage:           0%
Processes:            126
Users logged in:      1
IPv4 address for enp0s3: 192.168.56.11
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fea0:aa69

 * Ubuntu 20.04 LTS Focal Fossa has reached its end of standard support
   on 31 May 2025.

For more details see:
https://ubuntu.com/20-04

Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection
or proxy settings

Last login: Tue Jun  2 09:47:58 2026 from 192.168.56.10
student@worker1:~$ _
```

4. MPI Clustering Setup

OpenMPI and Python libraries were installed on all virtual machines.

A hostfile was created on the master node to define cluster resources and processing slots.

MPI enabled distributed execution of machine learning tasks across all nodes.

5. Distributed Jobs

Distributed machine learning jobs were executed using MPI.

Two distributed applications were implemented:

- MNIST digit classification
- Bioinformatics gene expression analysis

The dataset was divided among nodes, trained in parallel, and accuracy scores were gathered on the master node.

6. Docker Swarm

Docker was installed on all nodes to enable container orchestration.

The master node initialized the Docker Swarm cluster, and worker nodes joined using the generated swarm token.

Docker Swarm allowed distributed service deployment and management.

7. Apache Spark Cluster

Apache Spark was deployed using Docker Swarm.

The Spark environment included:

- Spark Master container
- Spark Worker containers

Deployment was managed through a Docker Compose stack configuration file.

8. Apache Spark Jobs

Distributed Spark jobs were implemented using PySpark.

Gene expression datasets were loaded into Spark, transformed into feature vectors, and processed using Random Forest classifiers.

Spark jobs were monitored using the Spark Web UI.

9. Monitoring and Spark UI

Spark jobs were monitored using the Spark Web Interface.

The interface displayed:

- Running jobs
- Worker node status
- Resource utilization
- Execution details
- Task completion statistics

Spark UI was accessed through port 8080.

10. Documentation and Submission

A complete project submission folder was prepared.

The folder included:

- Python scripts
- Spark configuration files
- Datasets
- Screenshots
- Setup logs
- Final report

The project directory was compressed into a ZIP archive for submission.

Conclusion

The project demonstrated practical implementation of virtualization, MPI clustering, Docker Swarm orchestration, Apache Spark processing, and distributed machine learning.